HASTIKA 2026 Task A: Character n-gram TF-IDF and Logistic Regression for Hate Speech Detection in Code-Mixed Kannada

Afsana Turabin Indana and H. M. Tahsin Sheikh and Humayra Jihan Arpita
University of Asia Pacific, Dhaka, Bangladesh
{22201235, 22201243, 22201244}@uap-bd.edu

Abstract

This paper describes our system for Task A (binary hate speech detection) of the HASTIKA shared task at ICON 2026, which asks systems to label Kannada–English code-mixed social media comments as *Hate* or *Non-Hate*. Our system combines character and word n-gram TF-IDF features with a regularised logistic regression classifier. In 5-fold cross-validation on the training data it reaches 0.811 macro-F1, and character n-grams are the most useful feature family (0.815 macro-F1 alone, against 0.757 for word features alone). We study the effect of regularisation, n-gram range, preprocessing and training-set size, and we analyse errors by hate category, comment length and model confidence. Hate comments in the Geopolitical, Religion and Political categories are missed most often (recall 0.73–0.74, against 0.85 for Gender), and half of all errors fall in the 22% of comments on which the model is least confident. The code is released to support reproduction.

1 Introduction

Social media platforms in India host a large amount of text in which several languages are mixed within one sentence and Indian languages are typed in the Latin script. Hate speech in such text is hard to detect for three reasons. First, romanised Kannada has no standard spelling, so the same word appears in many forms. Second, comments switch between Kannada and English, often within a clause. Third, abusive meaning is often carried by short, informal expressions and by references to people, groups and events that a model cannot see in the text alone.

The HASTIKA dataset (Kavatagi and Rachh, 2025) was built to study this problem for Kannada–English code-mixed comments. At the ICON 2026 shared task, Task A asks systems to classify each comment as *Hate* or *Non-Hate*.

We chose a deliberately simple and fully reproducible approach. Linear models over n-gram features are strong baselines for abusive language detection (Waseem and Hovy, 2016; Davidson et al., 2017; Wang and Manning, 2012), and character n-grams are a natural way to absorb the spelling variation of romanised text. Training takes seconds on a CPU, which allowed us to run a broad set of ablations. Our contributions are:

- a documented pipeline that reaches 0.811 macro-F1 in cross-validation, with released code;
- ablations over feature families, classifiers, regularisation strength, character n-gram range, preprocessing and training-set size; and
- an error analysis by hate category, comment length and model confidence, and an inspection of the features the model relies on.

2 Related Work

Hate speech detection. Surveys by Schmidt and Wiegand (2017) and Fortuna and Nunes (2018) describe how the field moved from lexicon and n-gram features to neural models, and note that surface features remain strong baselines. Waseem and Hovy (2016) found that character n-grams worked well on English Twitter data, and Davidson et al. (2017) showed that systems confuse hate speech with merely offensive language. Wang and Manning (2012) showed that simple n-gram models with linear classifiers are hard to beat on many text classification tasks.

Code-mixed Indian languages. Mathur et al. (2018) studied offensive tweets in Hinglish. For Dravidian languages, Hande et al. (2020) released KanCMD for Kannada–English sentiment and offensive language, and Chakravarthi et al. (2022) built DravidianCodeMix for Tamil, Malayalam and Kannada. HASTIKA (Kavatagi and Rachh, 2025)

Split	Comments	Hate	Non-Hate
Train	6,446	3,160	3,286
Validation (inputs only)	806	–	–

Table 1: Task A data. Gold labels for the validation inputs are not released.

adds hate speech and target labels for Kannada–English text. Pretrained multilingual encoders such as BERT (Devlin et al., 2019), XLM-R (Conneau et al., 2020) and MuRIL (Khanuja et al., 2021) are the usual stronger alternatives for such data. We did not use them (Section 9) and treat our system as a lexical baseline.

3 Task and Data

3.1 Task description

Task A is a binary classification task over single comments. Participants receive a labelled training file and an unlabelled validation file, and submit one label per validation comment. We also use the multiclass training file of the same dataset, which gives a category for each hate comment, but only for analysis and not for training.

3.2 Data statistics

Table 1 summarises the Task A files. The training set has 6,446 comments and is almost balanced (49.0% Hate). The validation file has 806 comments without labels.

Table 2 gives further statistics of the training set. Comments are short: the median is 8 words and 53 characters, and 11.8% have three words or fewer. Hate comments are longer than non-hate comments (12.8 against 9.6 words on average), so length alone carries some signal. The validation comments are similar in length (11.5 words on average). The text is noisy. 15.8% of training comments contain non-ASCII characters, mostly emoji, and some of these were mis-encoded in the source files. 7.2% contain HTML artefacts such as line-break tags and escaped quotes, and 2.0% contain user mentions.

The spelling variation of romanised text shows in the vocabulary. A quarter of all word tokens in the validation comments, and almost half of the word types, never occur in the training data. A word-level model has no information on these tokens, while a character-level model can still use their subword pieces. The training data has 42 duplicate comments in 35 groups. In three of these groups the duplicates carry different labels, which shows some annotation noise. We kept all duplicates.

Statistic	Value
Mean / median words per comment	11.2 / 8
Mean / median characters	75.4 / 53
Comments with $\leq$3 words	11.8%
Comments with non-ASCII characters	15.8%
Comments with HTML artefacts	7.2%
Vocabulary size (train / validation)	24,683 / 5,003
Validation word types unseen in train	45.5%
Validation word tokens unseen in train	25.4%
Duplicate comments in train	42

Table 2: Statistics of the training data after normalisation (Section 4.1). Vocabulary counts are whitespace-separated lowercased tokens.

Category	Comments	Share
Gender	1,362	43.1%
Political	559	17.7%
Others	449	14.2%
Religion	382	12.1%
Violence	221	7.0%
Geo-political	186	5.9%

Table 3: Hate categories in the multiclass training file.

The multiclass file (3,159 comments) assigns each hate comment one of six categories (Table 3). Gender is the largest category, with 43% of the comments.

4 System Description

4.1 Preprocessing

We decode HTML entities, remove HTML tags, replace URLs with a `URL` token and user mentions with `@user`, lowercase the text and collapse whitespace. We do not transliterate, stem, segment or remove stop words, because romanised Kannada has no consistent spelling and no standard tools. We leave emoji and mis-encoded character sequences unchanged. Character n-grams treat them as ordinary symbols, and a repeated mis-encoded emoji can still be informative.

4.2 Features

We concatenate two TF-IDF representations, both with sublinear term frequency $(1 + \log \mathrm{tf})$:

- character n-grams of length 2 to 5 taken inside word boundaries (`char_wb`), with a minimum document frequency of 2; and
- word unigrams and bigrams.

Each block is ℓ_2-normalised by the vectoriser.

Component	Setting
Character features	`char_wb`, 2–5, min_df 2
Word features	1–2 grams, min_df 1
Term weighting	TF-IDF, sublinear tf
Classifier	Logistic regression, ℓ_2
Regularisation	$C = 10$, max 2,000 iterations
Decision threshold	0.5
Cross-validation	Stratified, 5 folds, seed 42
Software	scikit-learn, pandas, NumPy

Table 4: Configuration of the system.

4.3 Classifier

The classifier is ℓ_2-regularised logistic regression from scikit-learn (Pedregosa et al., 2011) with C=10 and up to 2,000 iterations. We set these values by hand before running any experiment and did not tune them on the cross-validation folds. Section 5.3 shows how sensitive the results are to C. The final model is trained on all 6,446 training comments, and it labels a comment as *Hate* if the predicted probability exceeds 0.5. Table 4 lists the full configuration.

4.4 Reproducibility

The whole pipeline is one script (`run_taskA.py`) with a fixed seed (42). A second script (`extra_analysis.py`) produces the ablations in Section 5.3. Both need only pandas, NumPy, scikit-learn and Matplotlib, and a full run takes a few minutes on a CPU without a GPU. The code is available at `https://github.com/<username>/<repository>`.

5 Experiments

5.1 Setup

No gold labels are available for the validation comments, so we estimate performance with stratified 5-fold cross-validation on the training data (seed 42). We report accuracy and macro-F1, the mean over the five folds. Standard deviations over folds are between 0.005 and 0.010 for macro-F1. All models in one comparison use the same folds.

5.2 Feature families and classifiers

Table 5 compares five systems.

Character n-grams give a gain of about six points over word features. This is consistent with the vocabulary statistics above: a large part of the word vocabulary is unseen at test time. Adding word features to the character model does not help. The character-only model is 0.4 points higher, which is below one standard deviation across folds. Logistic regression and a linear SVM perform the same. We kept the combined model because we fixed it before the ablation, and we report the character-only result for transparency.

Model	Acc.	Macro-F1
Word TF-IDF + NB	.766	.765
Word TF-IDF + LR	.757	.757
Char TF-IDF + LR	**.815**	**.815**
Char+Word + LinearSVC	.811	.811
Char+Word + LR (ours)	.812	.811

Table 5: 5-fold cross-validation on the training set.

C	0.1	1	10	100
Macro-F1	.801	**.819**	.811	.808
Std.	.009	.008	.008	.010

Table 6: Effect of the regularisation strength C (Char+Word + LR, 5-fold cross-validation).

5.3 Ablations

Regularisation. Table 6 shows macro-F1 for different values of C with the final feature set. Performance peaks at C=1 (0.819) and is lower at our hand-set value C=10 (0.811), and strong regularisation hurts (C=0.1: 0.801). The 0.7-point gain from C=1 is about one standard deviation, so we cannot call it reliable, but it indicates that the system is slightly over-fitted at C=10.

Character n-gram range. Table 7 varies the range for a character-only model with C=10. Ranges that start at length 1 lose 1.7 points, and ranges that reach length 6 lose 0.8 points. The range 2–5 inside word boundaries is best. Crossing word boundaries (`char`) gives 0.811, which is no better.

Preprocessing. Our cleaning step (Section 4.1) changes the score very little: 0.8114 with cleaning and 0.8112 on the raw comments. The vectoriser already lowercases the text, so the difference comes only from removing HTML tags, URLs and mentions. These artefacts are rare (Table 2), and the model does not need the cleaning.

Training-set size. Figure 1a shows the macro-F1 when a fraction of each training fold is used. The score rises from 0.754 with 10% of the data to 0.801 with 50% and 0.811 with all data, and the curve is flattening. More data of the same kind would help

Character n-grams	Macro-F1
`char_wb` 1–3	.799
`char_wb` 2–4	.814
`char_wb` 2–5	**.815**
`char_wb` 3–6	.807
`char` 2–5	.811

Table 7: Character n-gram ranges (Char TF-IDF + LR, 5-fold cross-validation).

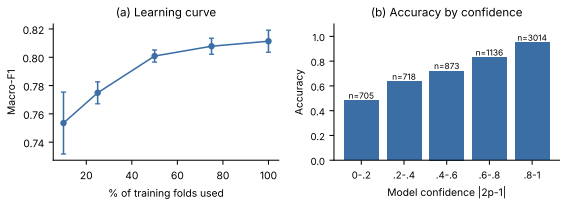


Figure 1: (a) Macro-F1 against the fraction of the training folds used; bars show one standard deviation over folds. (b) Out-of-fold accuracy by model confidence, where confidence is $|2p-1|$ for the predicted probability p of the Hate class.

only a little, so a gain would have to come from a better representation, such as a pretrained model, rather than from more n-gram features.

5.4 Detailed results of the final model

Pooling the out-of-fold predictions of the final model gives accuracy 0.812 and macro-F1 0.811. For the Hate class, precision is 0.812, recall 0.800 and F1 0.806. For Non-Hate, they are 0.811, 0.822 and 0.816. The errors are almost symmetric: 584 non-hate comments are labelled as hate and 631 hate comments are missed (Figure 2a).

6 Error Analysis

By hate category. We matched training comments to the multiclass file by id; 2,515 of the 3,160 hate comments have a category. Recall is highest for Gender (0.85, n=1,074) and Violence (0.80), and lowest for Geo-political (0.73), Religion (0.74), Political (0.74) and Others (0.75) (Figure 2b). A plausible reason, which we did not test, is that gender-directed abuse relies on recurring lexical patterns that n-grams capture, whereas political, religious and regional hate depends more on the target and the context. Gender is also the largest category, so the model sees most training examples of it.

By comment length. Table 8 shows accuracy by length. It is highest for comments of 1–3

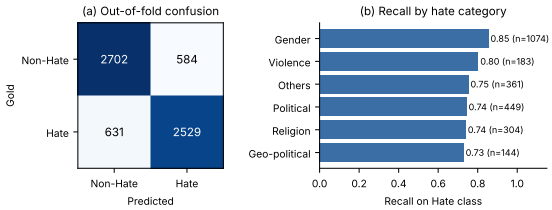


Figure 2: Out-of-fold results of the final model on the training data.

Words	Comments	Accuracy
1–3	758	.847
4–6	1,770	.836
7–12	2,177	.790
13–25	1,229	.806
>25	512	.779

Table 8: Out-of-fold accuracy by comment length.

words (0.85) and 4–6 words (0.84) and lower for longer comments (0.78–0.81). Short comments in this data often consist of a single abusive phrase, which is easy to match. We did not investigate the cause for longer comments, but they may mix argument, quotation and abuse, which a bag-of-n-grams model cannot separate.

By model confidence. Figure 1b shows that the predicted probability is informative. Accuracy is 0.95 on the 3,014 comments where the confidence is at least 0.8, and it falls to 0.48 (close to chance) on the 705 comments with confidence below 0.2. The 1,423 comments below 0.4 make up 22% of the data but hold 627 of the 1,215 errors (52%). A system could send such comments to a human moderator, or abstain on them, and keep a much higher accuracy on the rest.

Features. We fitted a word-unigram logistic regression on the training data (words with at least 5 occurrences) and inspected the 40 words with the highest weight for each class. We do not list them, for the reasons in the ethics statement. The Hate list consists almost entirely of abusive terms, insults aimed at the commenter's family, words of disrespect, and a small number of words that name a nationality or an ethnic or religious group. The Non-Hate list contains polite forms of address, English words such as praise and requests, and words about films, sport and everyday topics. So the model mostly learns the register of abuse. This works for explicit insults, and it explains why hate that is expressed without abusive words, such as the

political and regional categories, is missed more often.

7 Validation Predictions

The submitted validation file has 806 rows, and its ids match the validation inputs exactly. It predicts 391 Hate (48.5%) and 415 Non-hate comments, which is close to the 49.0% Hate share of the training data. Re-running the released script produces 394 Hate predictions, and 777 of the 806 labels (96.4%) agree with the submitted file. A second prediction file with 806 further comments, whose ids do not overlap the validation set, contains 402 Hate and 404 Non-Hate labels.

8 Discussion

The results support three observations. First, on this dataset the representation matters more than the classifier: logistic regression and SVM score the same, but character n-grams beat word features by six points. Second, the system is limited by what a lexical model can see. Performance depends on abusive vocabulary, the learning curve is flat, and the errors concentrate on comments for which the model has little evidence. Third, cheap model confidence is useful in practice, because it identifies a group of comments on which the model is close to guessing.

We expect a pretrained multilingual encoder to improve on our system, especially for the categories that depend on context. Whether it would also cope with the spelling variation of romanised Kannada is an open question that we could not test.

9 Limitations

We could not measure performance on the validation or test labels, so the numbers above are cross-validation estimates on the training data. The 42 duplicate comments may occur in both the training and the held-out folds, which gives a small optimistic bias. The C comparison and the n-gram comparison were run on the same folds, so the best setting in each table is also slightly optimistic. The model is a bag of n-grams: it has no notion of word order beyond bigrams, cannot resolve sarcasm and cannot recognise hate that uses no abusive words. We did not compare with pretrained multilingual transformers, which may do better. We analysed errors by category only for the 80% of hate comments that have a category label. Finally, our reading of the feature weights is qualitative.

10 Conclusion

A TF-IDF model over character n-grams with logistic regression is a strong, fast and reproducible baseline for hate speech detection in Kannada–English code-mixed comments. It reaches about 0.81 macro-F1 in cross-validation, and its errors are concentrated in low-confidence comments and in hate categories that depend on context. In future work we plan to fine-tune a multilingual encoder, to use the category labels as an auxiliary signal, and to combine the lexical model with an encoder.

Ethics Statement

The dataset contains abusive and offensive language directed at individuals and groups, and it includes references to real public figures. We quote none of it and we do not list the words that the model associates with hate, because such a list can be reused to produce abuse. Automatic hate detectors make errors in both directions: false positives can suppress legitimate political speech, and false negatives leave abuse in place. Because hate and non-hate comments differ in register, a classifier of this kind may also over-flag dialect or informal writing. A system like ours should support human moderators and not replace them.

Acknowledgments

We thank the HASTIKA organisers for the shared task and the annotated data.

References

Bharathi Raja Chakravarthi, Ruba Priyadharshini, Vigneshwaran Muralidaran, Navya Jose, Shardul Suryawanshi, Elizabeth Sherly, and John P. McCrae. 2022. DravidianCodeMix: Sentiment analysis and offensive language identification dataset for Dravidian languages in code-mixed text. *Language Resources and Evaluation*, 56(3):765–806.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451. Association for Computational Linguistics.

Thomas Davidson, Dana Warmsley, Michael Macy, and Ingmar Weber. 2017. Automated hate speech detection and the problem of offensive language. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 11, pages 512–515.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Paula Fortuna and Sérgio Nunes. 2018. A survey on automatic detection of hate speech in text. *ACM Computing Surveys*, 51(4):1–30.

Adeep Hande, Ruba Priyadharshini, and Bharathi Raja Chakravarthi. 2020. KanCMD: Kannada CodeMixed dataset for sentiment analysis and offensive language detection. In *Proceedings of the Third Workshop on Computational Modeling of People's Opinions, Personality, and Emotion's in Social Media*, pages 54–63, Barcelona, Spain (Online). Association for Computational Linguistics.

Sanjana Kavatagi and Rashmi Rachh. 2025. HASTIKA: hate speech and target identification in Kannada-English code-mixed text. *Language Resources and Evaluation*, 59(3):2811–2856.

Simran Khanuja, Diksha Bansal, Sarvesh Mehtani, Savya Khosla, Atreyee Dey, Balaji Gopalan, Dilip Kumar Margam, Pooja Aggarwal, Rajiv Teja Nagipogu, Shachi Dave, et al. 2021. MuRIL: Multilingual representations for Indian languages. *arXiv preprint arXiv:2103.10730*.

Puneet Mathur, Ramit Sawhney, Meghna Ayyar, and Rajiv Shah. 2018. Did you offend me? classification of offensive tweets in Hinglish language. In *Proceedings of the 2nd Workshop on Abusive Language Online*, pages 138–148, Brussels, Belgium. Association for Computational Linguistics.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Anna Schmidt and Michael Wiegand. 2017. A survey on hate speech detection using natural language processing. In *Proceedings of the Fifth International Workshop on Natural Language Processing for Social Media*, pages 1–10, Valencia, Spain. Association for Computational Linguistics.

Sida Wang and Christopher D. Manning. 2012. Baselines and bigrams: Simple, good sentiment and topic classification. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 90–94, Jeju Island, Korea. Association for Computational Linguistics.

Zeerak Waseem and Dirk Hovy. 2016. Hateful symbols or hateful people? predictive features for hate speech detection on Twitter. In *Proceedings of the NAACL Student Research Workshop*, pages 88–93, San Diego, California. Association for Computational Linguistics.